2026-05-05-p1-f11-session-transcript-resume

P1-F11 — Session Transcript Resume Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Persist session transcripts to disk; resume via `--resume` (project) / `--continue` (global) flags + `/sessions` slash + `helixcode sessions` cobra (list/show/resume/delete). JSONL transcripts in `$XDG_DATA_HOME/helixcode/sessions/<id>/`.

Architecture: New `internal/session/{resume.go, transcript_store.go, identity.go}`. Existing `session_manager.go` extended with `Append`, `Resume`, `CurrentID`. CLI flags parsed early in main.go, ResumeFinder resolves target, transcript replayed into session manager.

Tech Stack: Go 1.26, testify v1.11, gopkg.in/yaml.v3 (in go.mod), spf13/cobra, fsnotify, zap. **No new external deps.**

Spec: `docs/superpowers/specs/2026-05-05-p1-f11-session-transcript-resume-design.md` (commit `9128a9d`)

Working directory for `go` commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/session/resume.go internal/session/transcript_store.go
        internal/session/identity.go \
  internal/commands/sessions_command.go cmd/cli/sessions_cmd.go
        && echo BLUFF || echo clean
```

Task list

- [ ] P1-F11-T01 — bootstrap evidence + advance PROGRESS
- [ ] P1-F11-T02 — identity.go: computeProjectIdentity (Git root, cwd fallback) (TDD)
- [ ] P1-F11-T03 — transcript_store.go: JSONL append/read + metadata I/O (TDD)
- [ ] P1-F11-T04 — resume.go: ResumeFinder + ResumeMode + FindResumeTarget (TDD)
- [ ] P1-F11-T05 — session_manager extensions: Append + Resume + CurrentID (TDD)

- [ ] P1-F11-T06 — /sessions slash + helixcode sessions cobra (TDD)
- [ ] P1-F11-T07 — main.go: –resume/–continue flag parsing + integration test
- [ ] P1-F11-T08 — Challenge with runtime evidence (process restart preserves transcript)
- [ ] P1-F11-T09 — Feature 11 close-out + push to 4 remotes

Task 1: Bootstrap

Append F11 evidence section header (spec `9128a9d`), update PROGRESS current focus to F11, insert F11 task list (9 items) after F10's. Commit `docs(P1-F11-T01): bootstrap Phase 1 / Feature 11 evidence + advance PROGRESS`.

Task 2: identity.go (TDD)

Files: `internal/session/identity.go`, `internal/session/identity_test.go`.

Tests:

```
func TestComputeProjectIdentity_GitRoot(t *testing.T) {
    dir := t.TempDir()
    require.NoError(t, exec.Command("git", "-C", dir,
        "init").Run())
    sub := filepath.Join(dir, "sub", "deep")
    require.NoError(t, os.MkdirAll(sub, 0755))
    saved, _ := os.Getwd()
    defer os.Chdir(saved)
    require.NoError(t, os.Chdir(sub))
    id, err := ComputeProjectIdentity()
    require.NoError(t, err)
    // git rev-parse --show-toplevel returns the dir we
        initialised
    abs, _ := filepath.EvalSymlinks(dir)
    got, _ := filepath.EvalSymlinks(id)
    assert.Equal(t, abs, got)
}

func TestComputeProjectIdentity_NoGitFallback(t *testing.T) {
    dir := t.TempDir()
    saved, _ := os.Getwd()
    defer os.Chdir(saved)
    require.NoError(t, os.Chdir(dir))
    id, err := ComputeProjectIdentity()
    require.NoError(t, err)
```

```
    abs, _ := filepath.EvalSymlinks(dir)
    got, _ := filepath.EvalSymlinks(id)
    assert.Equal(t, abs, got)
}
```

Implementation:

```
package session

import (
    "os"
    "os/exec"
    "strings"
)

// ComputeProjectIdentity returns the Git toplevel for the cwd,
        or the cwd
// itself when not in a Git repo. Surfaces error only on os.Getwd
        failure.
func ComputeProjectIdentity() (string, error) {
    out, err := exec.Command("git", "rev-parse", "--show-
        toplevel").Output()
    if err == nil {
        return strings.TrimSpace(string(out)), nil
    }
    return os.Getwd()
}
```

TDD cycle. Subject: `feat(P1-F11-T02): identity.go: ComputeProjectIdentity (Git root + cwd fallback).`

Task 3: transcript_store.go (TDD)

Files: `internal/session/transcript_store.go`, `internal/session/transcript_store_test.go`.

Tests cover: AppendReadRoundTrip, HandlesCorruptedLine, MetadataResynth (delete metadata.json, re-derive), DeleteSession, ListSessionMetadata (project-scoped + global). Use real tempdirs.

Implementation: `TranscriptStore` struct with `baseDir string`. Methods: - `Append(ctx, sessionID, msg) error` — open `<base>/<id>/transcript.jsonl` with `O_APPEND|O_CREATE|O_WRONLY`, marshal msg, write line. - `ReadTranscript(ctx, sessionID) ([]Message, error)` — read line-by-line, skip malformed. - `ListSessionMetadata(ctx, projectPath) ([]SessionMetadata, error)` — walk `<base>/*/metadata.json`. If projectPath is empty, return all; else filter by metadata.ProjectPath equality. - `GetSessionMetadata(ctx, sessionID) (*SessionMetadata, error)` — read `<base>/<id>/metadata.json`. If missing, synthesize from transcript. - `UpdateSessionMetadata(ctx, meta) error` — write `<base>/<id>/`

`metadata.json`. - `DeleteSession(ctx, sessionID) error` — `os.RemoveAll(<base>/<id>)`.

`Message` is the existing type from internal/session — verify name/shape via `grep -n "^type Message" internal/session/*.go` first; ADAPT.

Subject: `feat(P1-F11-T03): transcript_store.go: JSONL append/read + metadata I/O`.

Task 4: resume.go — ResumeFinder (TDD)

Files: `internal/session/resume.go`, `internal/session/resume_test.go`.

Tests: - `FindMostRecentInProject` (multiple sessions, latest LastActivity wins) - `FindMostRecentGlobal` - `NoSessions` returns descriptive error - `Resume_LoadsTranscript`

Implementation:

```go
package session

import (
    "context"
    "fmt"
    "sort"
    "time"
)

type ResumeMode string
const (
    ResumeProject ResumeMode = "project"
    ResumeGlobal  ResumeMode = "global"
)

type SessionMetadata struct { /* per spec */ }

type SessionStore interface {
    ListSessionMetadata(ctx context.Context, projectPath string)
        ([]SessionMetadata, error)
    GetSessionMetadata(ctx context.Context, sessionID string)
        (*SessionMetadata, error)
    UpdateSessionMetadata(ctx context.Context, meta
        SessionMetadata) error
    Append(ctx context.Context, sessionID string, msg Message)
        error
    ReadTranscript(ctx context.Context, sessionID string)
        ([]Message, error)
    DeleteSession(ctx context.Context, sessionID string) error
}
```

```
type ResumeFinder struct{ store SessionStore }

func NewResumeFinder(store SessionStore) *ResumeFinder { return
        &ResumeFinder{store: store} }

func (rf *ResumeFinder) FindResumeTarget(ctx context.Context,
        mode ResumeMode, currentProject string)
        (*SessionMetadata, error) {
    var lookup string
    if mode == ResumeProject {
        lookup = currentProject
    }
    metas, err := rf.store.ListSessionMetadata(ctx, lookup)
    if err != nil { return nil, fmt.Errorf("list sessions: %w",
        err) }
    if len(metas) == 0 { return nil, fmt.Errorf("no sessions
        found to resume") }
    sort.Slice(metas, func(i, j int) bool { return
        metas[i].LastActivity.After(metas[j].LastActivity) })
    return &metas[0], nil
}

func (rf *ResumeFinder) Resume(ctx context.Context, sessionID
        string) ([]Message, *SessionMetadata, error) {
    meta, err := rf.store.GetSessionMetadata(ctx, sessionID)
    if err != nil { return nil, nil, fmt.Errorf("metadata: %w",
        err) }
    msgs, err := rf.store.ReadTranscript(ctx, sessionID)
    if err != nil { return nil, meta, fmt.Errorf("transcript:
        %w", err) }
    return msgs, meta, nil
}
```

Subject: `feat(P1-F11-T04): resume.go: ResumeFinder + ResumeMode + FindResumeTarget`.

Task 5: SessionManager extensions (TDD)

Find existing `internal/session/session_manager.go`. Add: - `Append(ctx, msg) error` — calls `store.Append(ctx, m.currentID, msg)` + updates metadata (LastActivity, MessageCount++) - `Resume(ctx, id) error` — `store.GetSessionMetadata` + `store.ReadTranscript`, replay into in-memory session state, set currentID - `CurrentID() string` — returns active session ID

Tests: `Append_PersistsThroughStore`, `Resume_LoadsTranscript`, `CurrentID_ReturnsActive`.

Subject: `feat(P1-F11-T05): SessionManager.Append/Resume/CurrentID`.

Task 6: /sessions slash + helixcode sessions cobra (TDD)

Mirror F09/F10 surface patterns. Tests cover list/show/resume/delete/unknown.

Implementation `internal/commands/sessions_command.go` + `cmd/cli/sessions_cmd.go`. Both surfaces share a `SessionStore`.

Subject: `feat(P1-F11-T06): /sessions slash + helixcode sessions cobra`.

Task 7: main.go startup wiring + integration test

In `cmd/cli/main.go`: 1. Add `--resume` / `--continue` / `--resume <id>` flag parsing **before** the existing dispatcher. 2. Resolve project identity, look up target via ResumeFinder. 3. Inject resumed transcript into session manager. 4. Wire `/sessions` slash + cobra dispatcher.

Integration tests `tests/integration/sessions_resume_test.go`: - `TestSessions_ResumePersistsAcrossRestart` — write 3 messages via real file I/O, simulate restart, assert all 3 readable. - `TestSessions_GlobalFindsMostRecentAcrossProjects` — two project dirs, two sessions, global finder returns the more recent.

Subject: `feat(P1-F11-T07): wire resume into main.go + integration test`.

Task 8: Challenge with runtime evidence

Harness `tests/integration/cmd/p1f11_challenge/main.go`: 1. Start session in tempdir, append 3 messages 2. Read metadata, verify MessageCount=3 3. Simulate restart (drop session manager, construct new with same store + sessionID) 4. Resume; assert all 3 messages restored 5. Test global resume finds the session across project boundaries 6. Anti-bluff smoke clean

run.sh + CHALLENGE.md in `challenges/p1-f11-session-resume/`. Dual commit. Verbatim evidence in `06_phase_1_evidence.md`.

Subject: `feat(P1-F11-T08): challenge with runtime evidence + cross-compile check`.

Task 9: Close-out + push

Tick 9 items, advance PROGRESS to idle (F12 candidate), final verification, commit, push 4 remotes non-force.

Self-review notes

1. Spec coverage: each spec section maps to a task (T02 identity, T03 store, T04 finder, T05 manager extensions, T06 surfaces, T07 wiring, T08 challenge, T09 close-out).
2. TDD: every code task starts with failing tests.
3. Type consistency: `SessionMetadata`, `ResumeMode`, `ResumeFinder`, `SessionStore`, `TranscriptStore`, `ComputeProjectIdentity`, `Message` consistent.
4. Cross-platform: pure Go + os/exec for `git rev-parse`; fsnotify not needed (resume is request-response, not event-driven).
5. Anti-bluff: full 4-term smoke + Challenge captures real process-restart resumption.
6. No new deps.
7. Branch + push: stays on main, non-force to all four remotes.